

Machine Learning 6

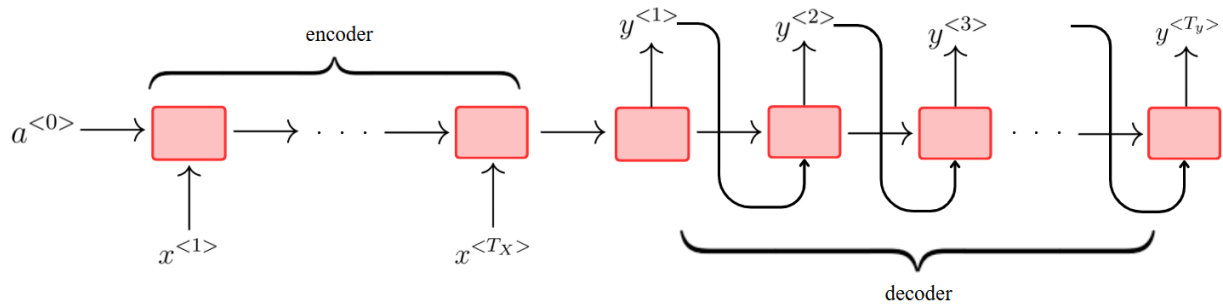
Sequence Models and Attention Mechanisms

Sequence-to-Sequence Model

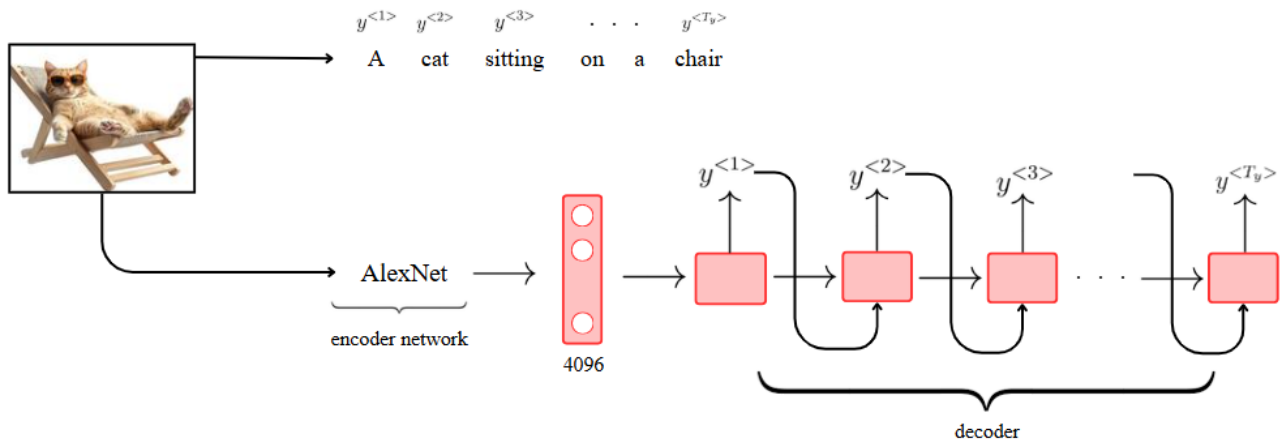
Example: Language Translation

$x^{<1>}$	$x^{<2>}$	$x^{<3>}$	$x^{<4>}$	$x^{<5>}$	
Jane	visite	l'Afrique	en	septembre	
Jane	is	visiting	Africa	in	September
$y^{<1>}$	$y^{<2>}$	$y^{<3>}$	$y^{<4>}$	$y^{<5>}$	$y^{<6>}$

Let's build an **encoder-decoder** network (could be GRU/LSTM) (for French words)

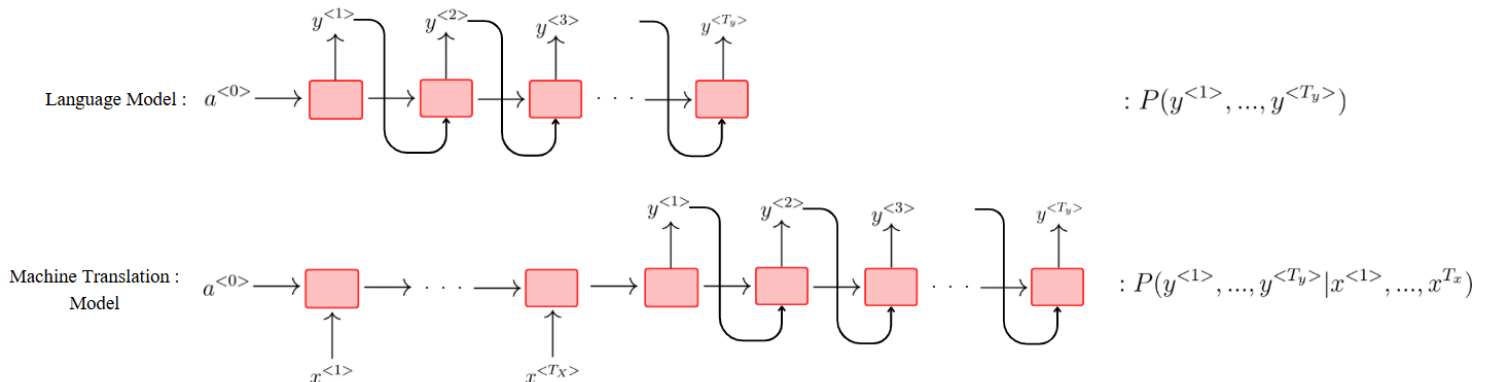


Example: Image Captioning



Note: Here the encoder network gets the image, this setup works well for image captioning as long as the caption isn't too long.

Note: We can view the task of building a machine-language-translation as building a **conditional language model**



Note: How the decoder network in the Machine-Translation-Model is similar to the Language Model. This is why the Machine-Translation Model is aka "Conditional Language Model", since it is trying to evaluate what is the probability of an English sentence given the French sentence (for example).

Example:

"Jane visite l'Afrique en septembre"

$$\rightarrow P(\underbrace{y^{<1>}, y^{<2>}, \dots, y^{<T_y>}}_{\text{English}} | \underbrace{x}_{\text{French}})$$

Note: It would be a bad idea to sample words at random because we may get some very wacky translations. Instead we use,

$$\arg_{y^{<1>}, y^{<2>}, \dots, y^{T_Y}} \max P(y^{<1>}, y^{<2>}, \dots, y^{<T_y>} | x)$$

Why not just use Greedy Search

You could just pick the 1st best pick for the 1st word, first pick for 2nd and so on, but because English has some words appear more commonly such as "going" or "visiting" the model is likely to suggest a translation of,

$$P(\text{"Jane is going"} | x) > P(\text{"Jane is visiting"} | x)$$

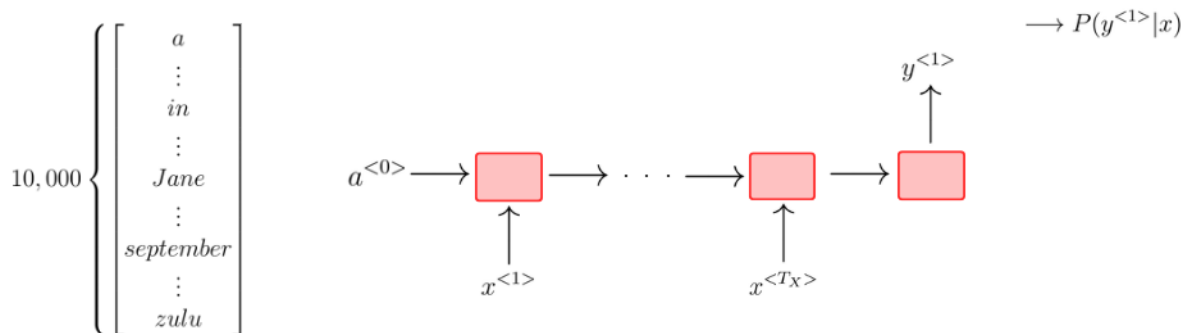
which is inaccurate. This unfortunately is an example of a much broader phenomenon in Machine Learning.

Beam Search Algorithm

Example:

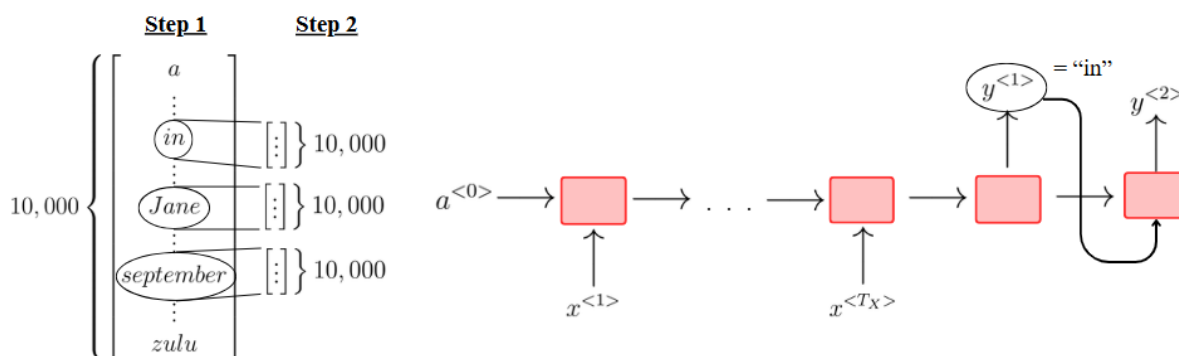
B = 3 (example): To say that Beam which will consider not just one possibility but 3 at a time.

Step 1



Step 2

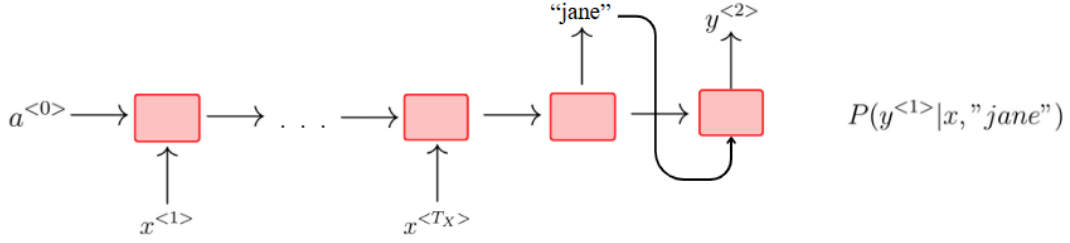
Having picked 3 options from step 1 ("in", "Jane", "september").



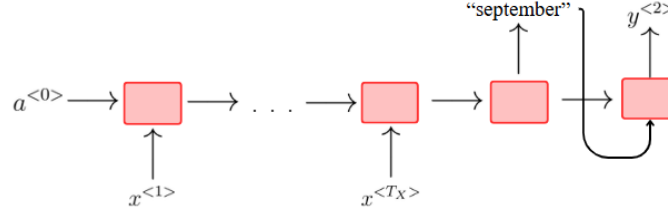
where,

$$\begin{aligned}
 &P(y^{<2>} | x, \text{"in"}) \\
 &\quad \downarrow \\
 &P(y^{<1>}, y^{<2>} | x) = P(y^{<1>} | x) \cdot P(y^{<2>} | x)
 \end{aligned}$$

you would get 3 options and you have to do the same thing for "jane".



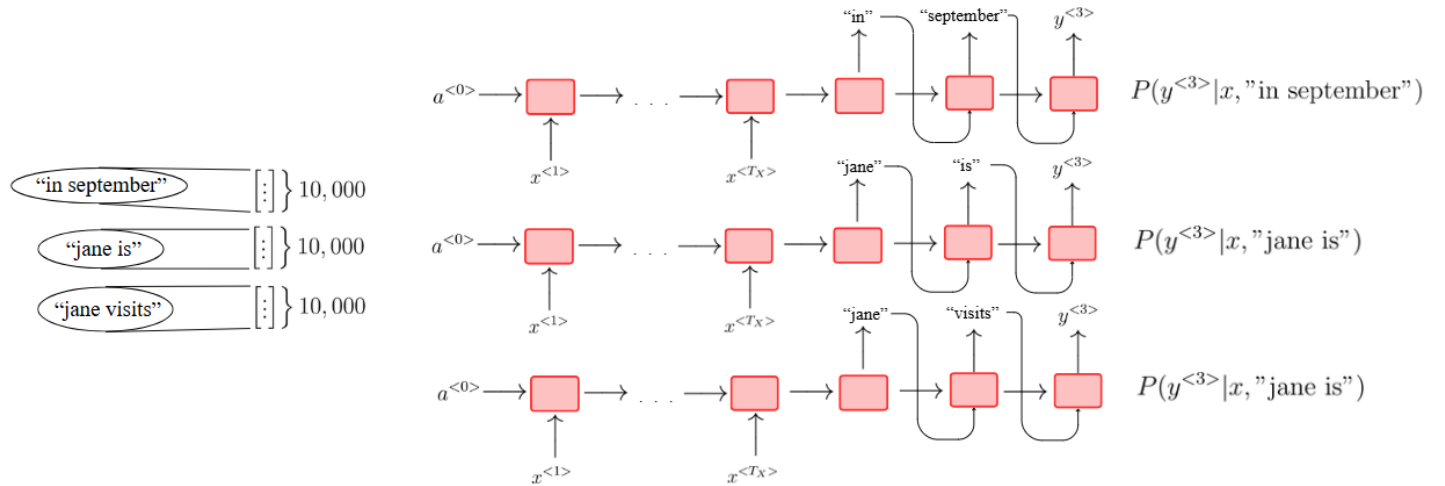
and the same for "september",



With all the results that we get we will only pick the top 3 ($B = 3$)

- "in september"
- "jane is"
- "jane visits"

Next, $B = 3$,



and again, we pick up the top 3.

Hopefully if this process is continued, the outcome will be,

"Jane visits africa in september iEOS_i"

Note: If $B = 1$, then this becomes the greedy search algorithm

Length Normalisation

$$\arg \max_y \prod_{t=1}^{T_y} P(y^{<t>}|x, y^{<1>}, \dots, y^{<t-1>})$$

Note: This is just a more general expression of the fact that,

$$P(y^{<1>}, \dots, y^{<T_y>}|x) = P(y^{<1>}|x) \cdot P(y^{<2>}|x, y^{<1>}) \dots P(y^{<T_y>}|x, y^{<1>}, \dots, y^{<T_y-1>})$$

Note: By taking a sum of logs instead we end up with a more numerically stable algorithm. Less prone to numerical rendering errors (underflow). Also, mathematically $\max P(y^?)$ should give same result as $\log P(y^?)$ as they are both monotonic functions.

$$\implies \arg \max_y \sum_{t=1}^{T_y} \log P(y^{<t>}|x, y^{<1>}, \dots, y^{<T_y-1>})$$

Note: This objective function has a rather undesirable effect that it naturally prefers shorter (more concise) translations over larger (possibly more accurate) ones. As fewer than one numbers multiplied would result in a greater quantity.

Note: We can also normalise this by the number of words in our translation. This patches the above mentioned issue,

The stricter approach,

$$\frac{1}{T_y} \sum_{t=1}^{T_y} \log P(y^{<t>} | x, y^{<1>}, \dots, y^{<T-1>})$$

or

The softer approach,

$$\left[\frac{1}{T_y^\alpha} \right] \sum_{t=1}^{T_y} \log P(y^{<t>} | x, y^{<1>}, \dots, y^{<T-1>})$$

(no normalisation) no compensation = 0 < y < 1 = total compensation (full normalisation)

Different values of B ?

- Large B: better result, slow + expensive
- Small B: lesser result, fast + expensive

Note: 3 is on the smaller size, prediction systems are typically 10, research operations do push this into the thousands. Diminishing returns as B gets very large.

Note: Unlike exact search algorithms such as BFS (or DFS) Beam search runs faster but is not guaranteed to find exact maximum for $\arg \max_y P(y|x)$

Error Analysis

Example:

”Jane visite l’Afrique em Septembre”

- Human Translation: Jane vists Africa in September (y^*)
- Algorithm Translation: Jane visited Africa last September ($\hat{y}$)

Note: The poor translation of the algorithm may be blamed on both the RNN or the Beam-parameter: B

Note: We can try to check this by computing,

$$P(y^*|x) \gtrless P(\hat{y}|x)$$

- Case I: $P(y^*|x) > P(\hat{y}|x)$
 - Beam search chose $\hat{y}$. But y^* attains higher $P(y|x)$.
 - Conclusion: Beam search is at fault.
- Case I: $P(y^*|x) \leq P(\hat{y}|x)$
 - y^* is a better translation than $\hat{y}$. But RNN predicted $P(y^*|x) \leq P(\hat{y}|x)$
 - Conclusion: RNN model is at fault.

Error Analysis Process

Human	Algorithm	$P(y^* x)$	$P(\hat{y} x)$	At fault?
”Jane visits Africa in September”	”Jane visited Africa last September”	2×10^{-10}	1×10^{-10}	B
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Note: Figures out what fraction of errors are ”due to” beam-search vs. RNN-model.

Evaluating Machine Translation

Example:

[French]: Le chat est sur le tapis.

reference 1: The cat is on the mat.

reference 2: There is a cat on the mat.

Machine Translation: "The the the the the the the" (bad translation)

Precision: $\frac{7}{2}$ (you look at each word of the the machine translation and try to see if it occurs even once in the reference-1,2)

Modified Precision: $\frac{2}{7}$ (the maximum numbers of this precision happens in a sentence, reference 1 had 2 - "the")

Bleu

Bilingual Evaluation understudy

Bleu-score on bigrams

Note: Can also have trigrams (3 words ...)

Example:

Reference-1: "The cat is on the mat"

Reference-2: "There is a cat on the mat"

Machine Translation: "The cat the cat on the mat"

distinct bigrams: { "the cat" "cat the" "cat on" "on the" "the mat" }
count: { 2 1 1 1 1 }

Distinct bigrams	Count	Count _{clip}
"the cat"	2	1
"cat the"	1	0
"cat on"	1	1
"on the"	1	1
"the mat"	1	1
$\sum \text{count} = 6$		$\sum \text{count}_{\text{clip}} = 4$

$$\Rightarrow \frac{\text{Bigram}_{\text{Count}_{\text{clip}}}}{\text{Bigram}_{\text{count}}} = \frac{4}{6} = \frac{2}{3}$$

Note: Earlier what we had done would have been referred to as uni grams

Example:

We can take the machine translation,

"the cat the cat on the mat" - $\hat{y}$

against,

Reference-1: "The cat is on the mat"

Reference-2: "There is a cat on the mat"

Unigram-bleu-score

$$P_1 = \frac{\sum_{\text{unigrams} \in \hat{y}} \text{Count}_{\text{clip}}(\text{unigram})}{\sum_{\text{unigram} \in \hat{y}} \text{Count}(\text{unigram})}$$

More generally,

N-gram bleu score

$$P_n = \frac{\sum_{n\text{-grams} \in \hat{y}} \text{Count}_{clip}(n\text{-gram})}{\sum_{n\text{-gram} \in \hat{y}} \text{Count}(n\text{-gram})}$$

Bleu Details

P_n = Bleu score on n-grams only

$$\text{Combined bleu-score} : BP \exp \left(\frac{1}{4} \sum_{n=1}^4 P_n \right)$$

BP: Brevity Penalty

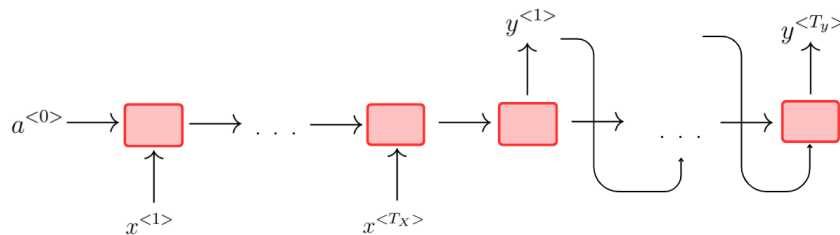
Note: It turns out if you output very short translations its easier to get high precision because probably most of the words you output appear in the references. BP is an adjustment factors which penalises translations that are too short.

$$BP = \begin{cases} 1 & \text{if machine-translation-output-length} > \text{reference-output-length} \\ \exp \left(1 - \frac{\text{reference-output-length}}{\text{machine-translation-output-length}} \right) & \text{if otherwise} \end{cases}$$

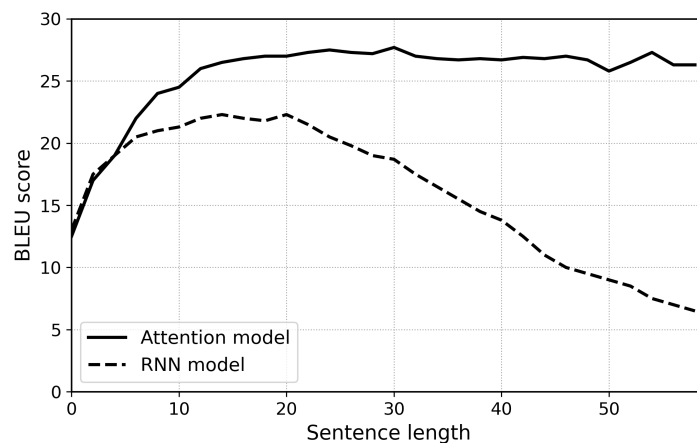
Attention

The problem with long sequences ?

Example:

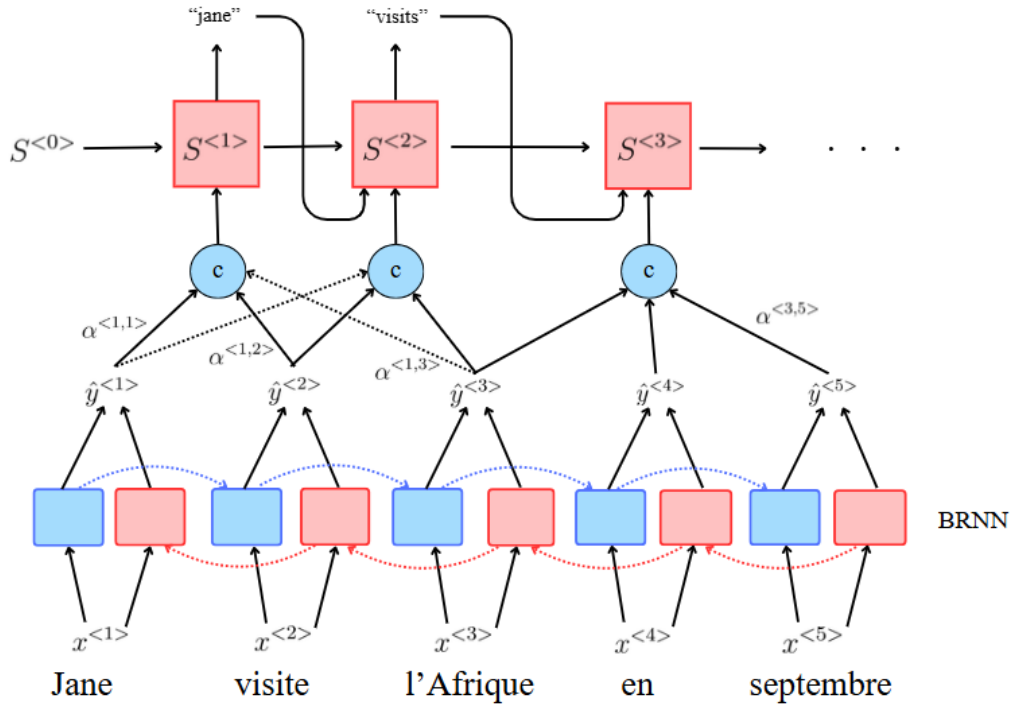


”Jane s’est rendue en Afrique en septembre dernier, a apprécié la culture et a rencontré beaucoup de gens merveilleux. Elle est revenue en racontant à quel point son voyage était merveilleux, et elle me tente d’y aller aussi.



Intuition

Example:

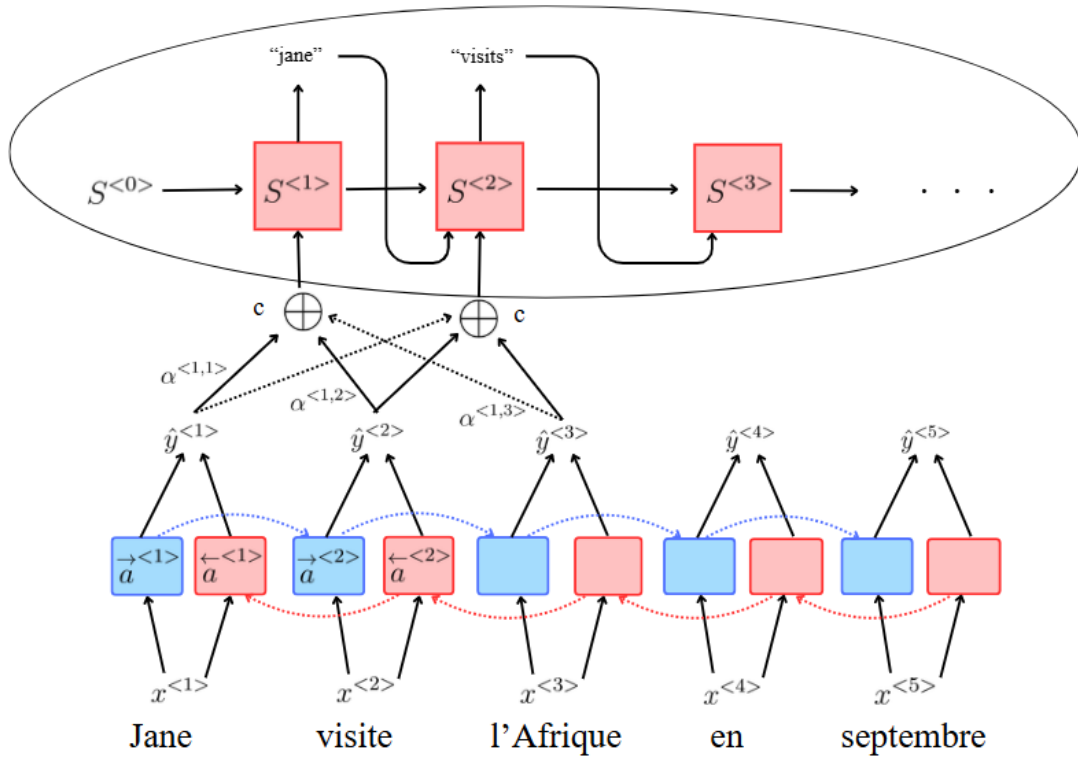


Note: $S^{<0>}$ here is the activation functions, earlier denoted by $a^{<0>}$. Therefore called States.

Note: $\alpha^{<1,1>}$ and $\alpha^{<1,2>}$ are attention terms which instruct the RNN on top as to how much attention should it pay to the first and second words of the sentence.

Note: c is the context which is, contributed by attention.

Attention Model



$\alpha^{<t,t'>}$ = amount of "attention" $y^{<t>}$ should pay to $a^{<t'>}$

$$\rightarrow c^{<1>} = \sum_t \alpha^{<1,t'>} a^{<t'>}$$

where,

$$a^{<t'>} = (\overleftarrow{a}^{<t'>}, \overrightarrow{a}^{<t'>})$$

we can also show that,

$$\sum_{t'} \alpha^{<1,t'>} = 1$$

there are obviously multiple contexts, c,

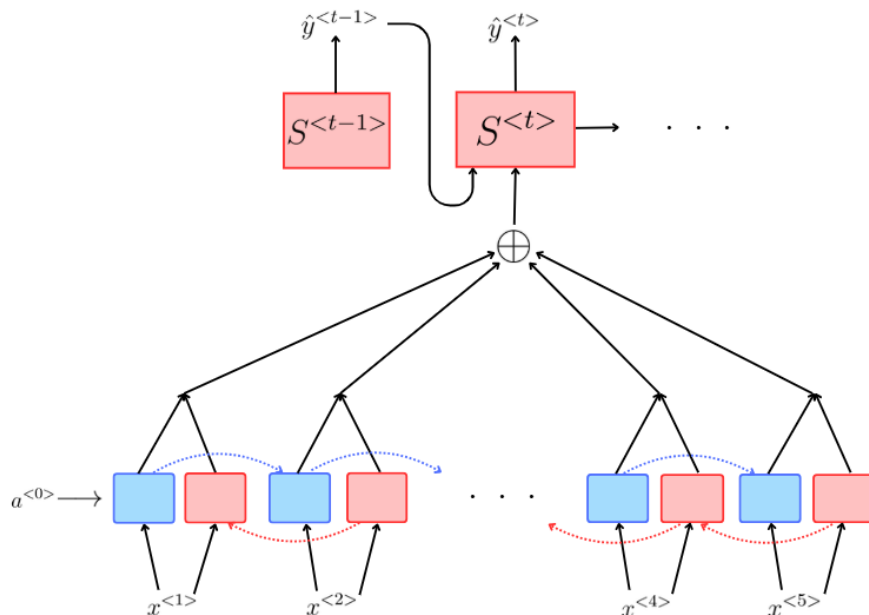
$$\begin{aligned} \rightarrow c^{<2>} &= \sum_{t'} \alpha^{<2,t'>} a^{<t'>} \\ \rightarrow c^{<3>} &= \dots \\ &\vdots \end{aligned}$$

Computing Attention $\alpha^{<t,t'>}$

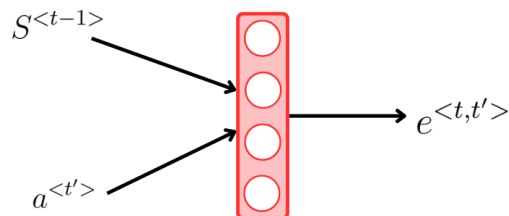
$\alpha^{<t,t'>}$ = amount of attention $y^{<t>}$ should pay to $a^{<t'>}$

$$\rightarrow \alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$

This is usually calculated in the following setup,



and,



Example:

An application could be to standardise dates received in inconsistent formats,

July 20th 1969 $\rightarrow$ 1969 – 07 – 20

23rd April, 1564 $\rightarrow$ 1564 – 04 – 23

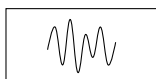
Speech Recognition Problem

Example:

$X \longrightarrow Y$

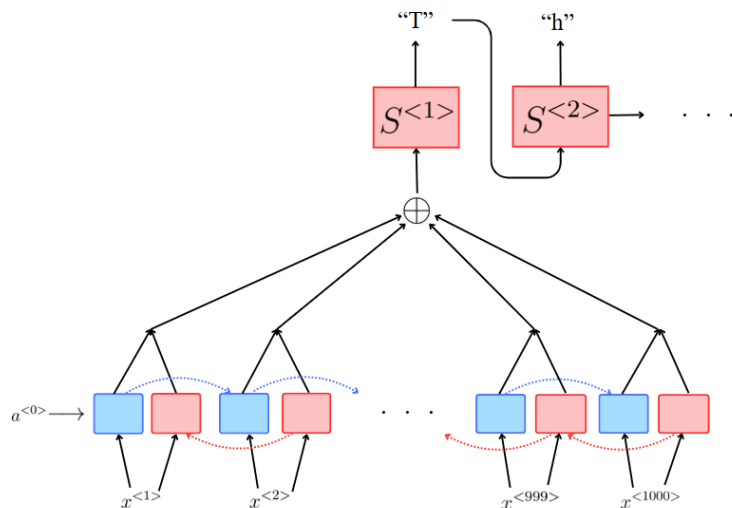
audio clip

transcript



“the quick brown fox”

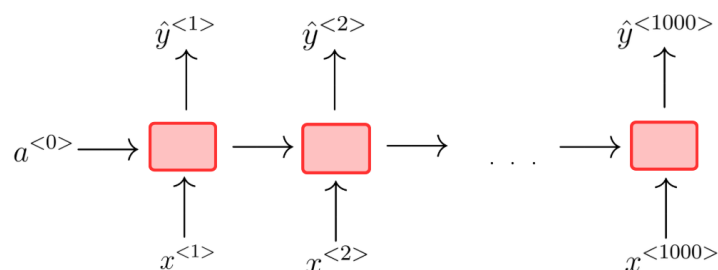
Attention Model for Speech Recognition



CTC cost for speech recognition

CTC: Connections-Temporal-Classification

"the quick brown fox"



Note: In-practice RNNs that input audio and output text have a really high number of input / time steps and significantly error output time steps.

Example:

Example of correct output for the first part of the earlier sentence,

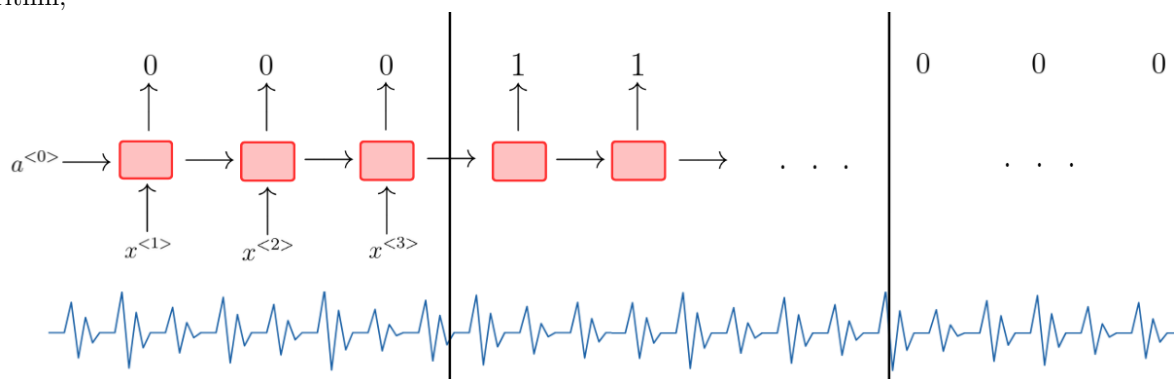
"t t t _ h _ e e _ _ _ _ q q q _ _" $\rightarrow$ "the_q"

Note: where _ is blank and _ is a "space" character

Trigger Word Detection

Example: Alexa (Amazon)

The algorithm,



Note: Important solution to the problem of not being able to detect small 1 pulse: make it output a string of 1's.

Note: But creating a dataset like this creates a very unbalanced set, as it contains a lot more 0's than 1's.